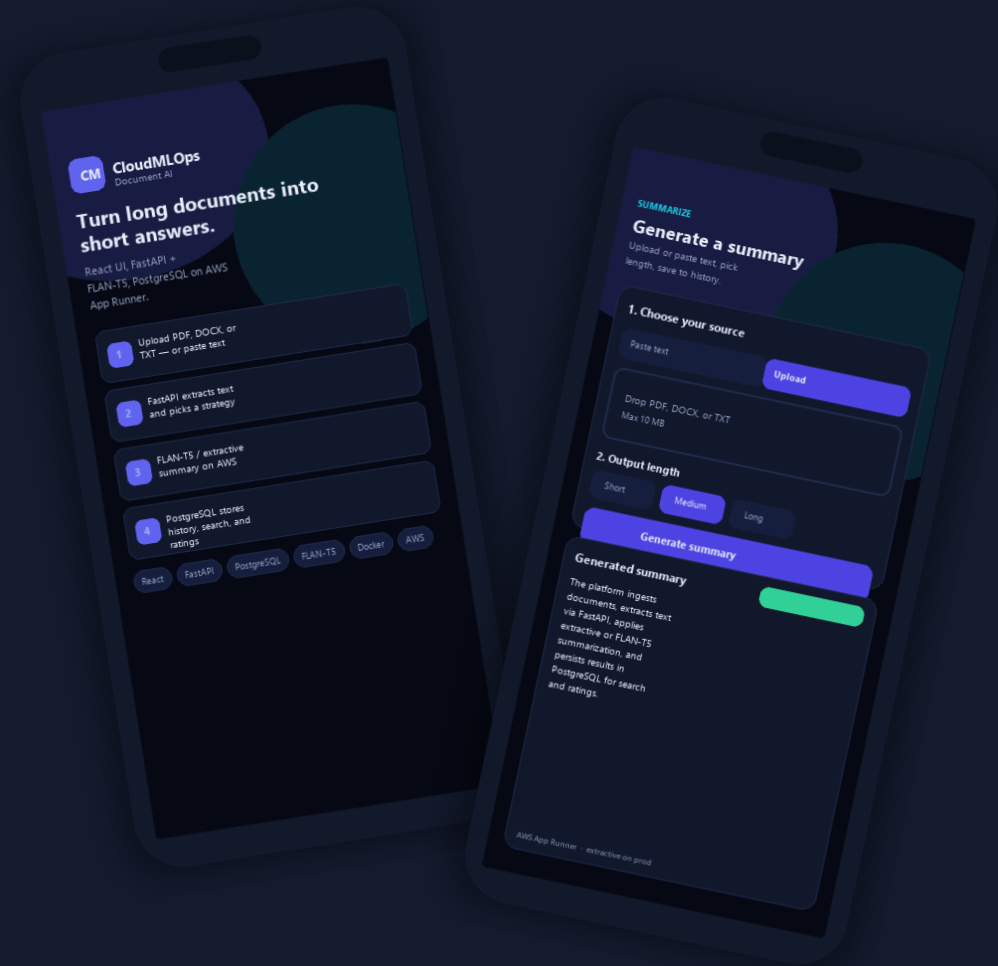


CLOUDEMLOPS

AI Document Summarization Platform

- Ingest PDF, DOCX, and TXT in one workspace
- FastAPI + FLAN-T5 / extractive summarization
- History, ratings, and admin MLOps on AWS
- Live on App Runner — demo-ready URLs in-deck



Project Overview

Smart summarization + cloud MLOps in one product

171

Automated tests

3

Architecture tiers

6

Live URLs

AWS

Production

- Upload PDF / DOCX / TXT — or paste text — get a concise summary in seconds.
- Every summary is saved: search history, download, and rate quality.
- Admins track usage, manage users, and approve new model versions.
- Shipped with Docker, GitHub Actions, Terraform, and AWS App Runner.

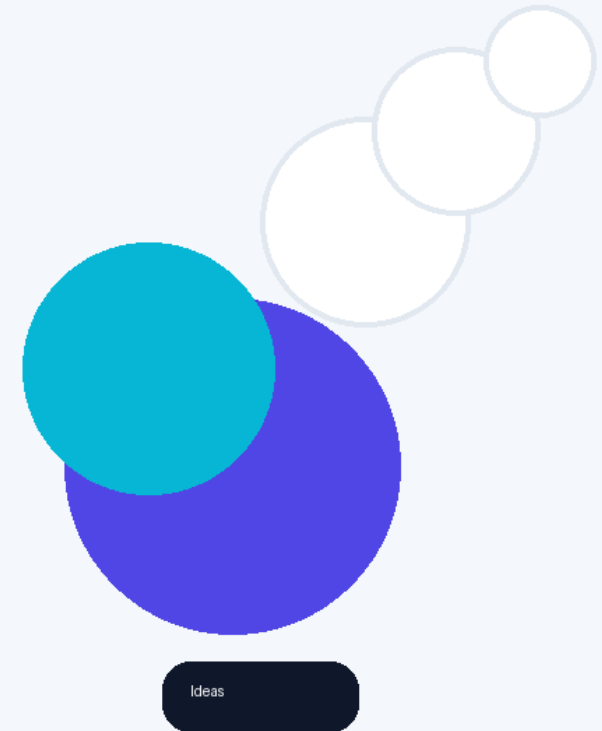


PDF DOCX TXT

Problem Statement

Why manual reading does not scale

- Reports and papers are too long to read end-to-end under time pressure.
- Notes in Word or email are not searchable or shared safely.
- Heavy AI models stall on free tiers and HTTP timeouts in the cloud.
- Teams need one secure app: extract, summarize, store, and deploy reliably.



Objectives

What we set out to build

Three-tier design

React + FastAPI + PostgreSQL

Smart ingestion

PDF, DOCX, TXT with validation

AI engine

FLAN-T5 locally; fast extractive on AWS

User experience

Auth, history, downloads, ratings

Admin & MLOps

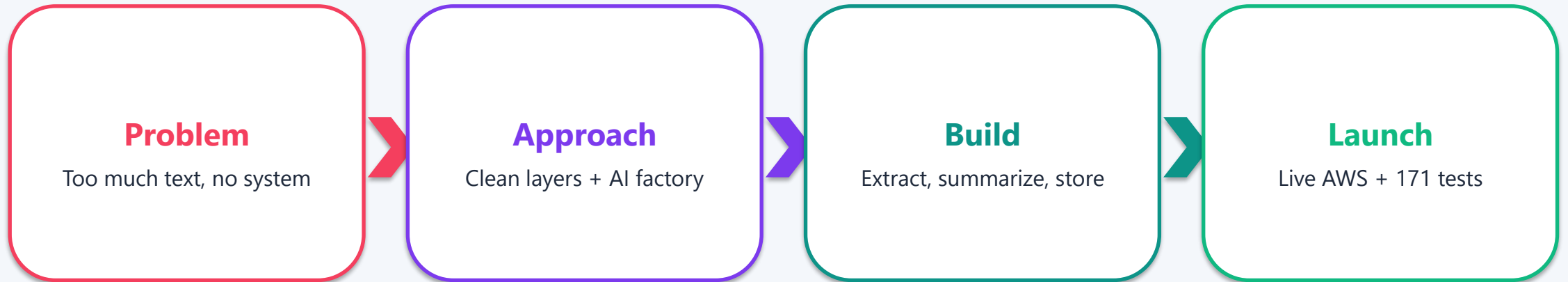
Metrics + model approve / promote

Cloud ship

CI/CD to App Runner via ECR

Proposed Solution

From document to deployed insight



Ship to Production

End-to-end flow

Users upload or paste documents → FastAPI validates and extracts text → summarizer (171 automated tests) persists output to PostgreSQL → React UI and admin dashboard expose history, ratings, and model lifecycle on AWS App Runner.

Technologies Used

Modern stack — only what the project uses

Frontend

React 19, Vite, React Router, Axios

Backend

Python 3.12, FastAPI, SQLAlchemy 2

Database

PostgreSQL 16, Alembic

AI / ML

Transformers, flan-t5-small, extractive

Extraction

PyMuPDF, python-docx

Cloud / DevOps

Docker, Actions, ECR, App Runner, Terraform

System Architecture

How traffic flows in production

Browser / User

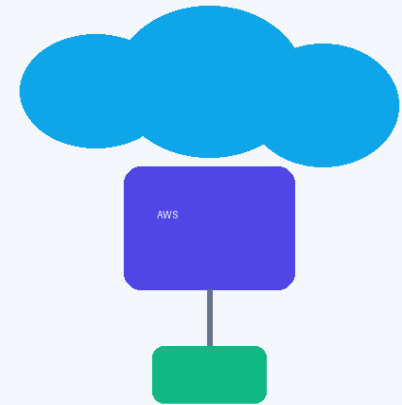
React frontend — AWS App Runner

FastAPI backend — AWS App Runner

PostgreSQL (RDS) + document storage

CloudWatch + Secrets Manager

CI/CD — GitHub Actions → ECR → deploy



Key Features

What users and admins can do

Authentication

JWT sessions, admin roles

Summarization

Text + documents, 3 lengths

History

Search and pagination

Feedback

Star ratings for quality

Admin dashboard

Usage and user management

Model lifecycle

UC-12 / UC-16 ROUGE pipeline

Interactive Demo

Click buttons in Slide Show (F5) to open live services

Try the app while you present:

1. Open AWS frontend and log in
2. Upload a sample PDF or paste text
3. Show summary + history
4. Optional: Swagger /docs for API



Launch Web App

Open Swagger UI

Health Check JSON

View Source on GitHub

Presenter tip: keep this slide open during Q&A — every button opens the real production service.

Live Deployment URLs

Interactive: click any link or button (Slide Show mode)

Tip: Press F5 in PowerPoint, then click highlighted links to open each service in your browser.

Open AWS App

Open API Docs

Open GitHub

AWS production UI

<https://asqmhswdfs.us-east-2.awsapprunner.com>

AWS production API

<https://p3jivcdmbf.us-east-2.awsapprunner.com>

Health check

<https://p3jivcdmbf.us-east-2.awsapprunner.com/health>

Swagger API docs

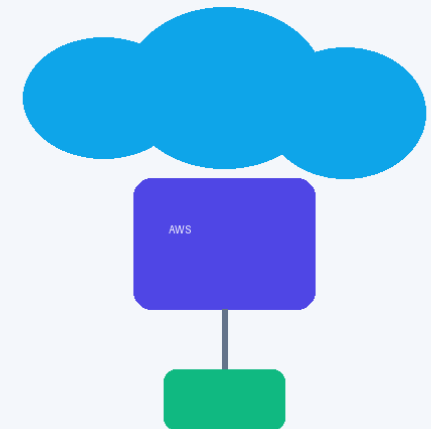
<https://p3jivcdmbf.us-east-2.awsapprunner.com/docs>

ReDoc API docs

<https://p3jivcdmbf.us-east-2.awsapprunner.com/redoc>

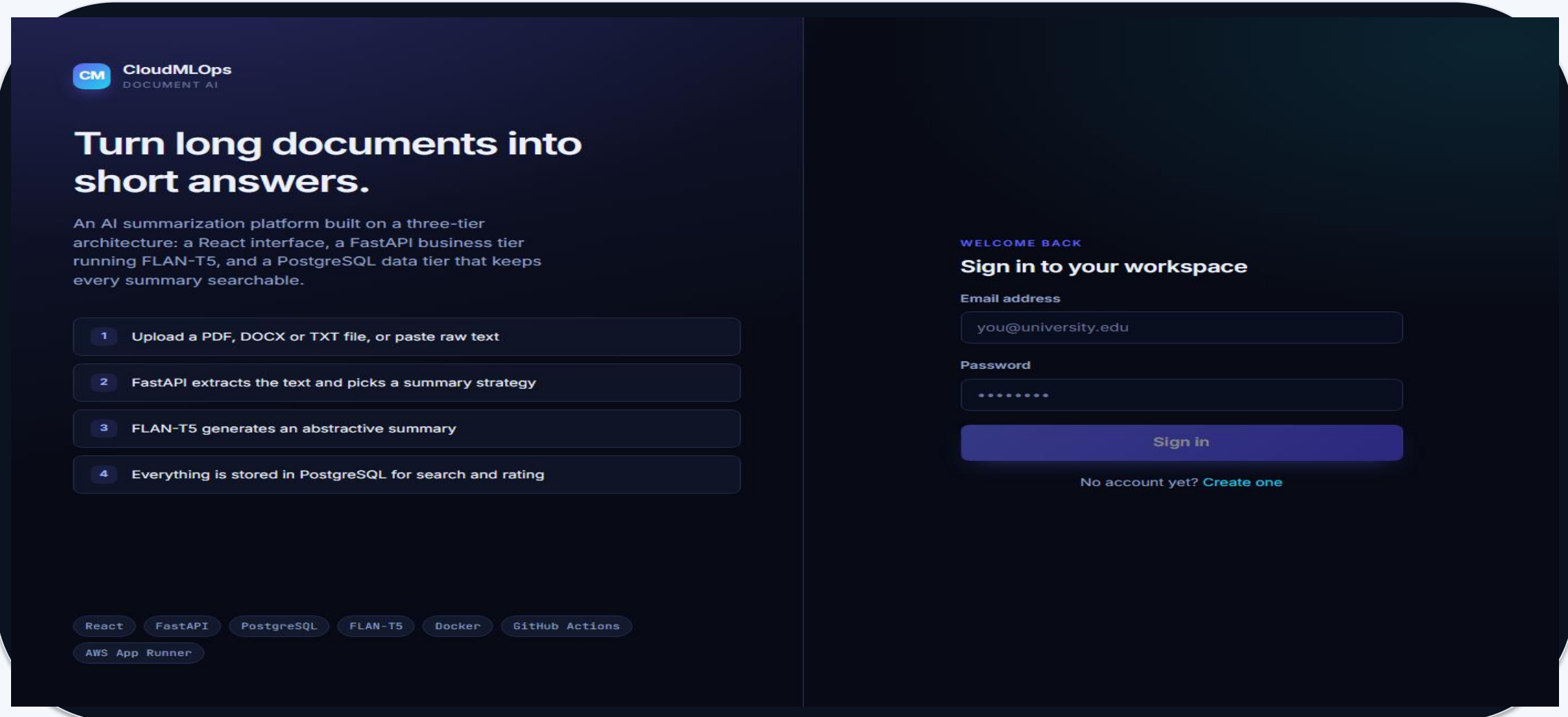
GitHub repository

<https://github.com/nitya200/CloudMLOps>



Production Application

AWS App Runner frontend



Live CloudMLOps UI on AWS (home / login route).

API Documentation

Interactive OpenAPI on production API

CloudMLOps Document Summarizer 1.0.0 OAS 3.1

[/openapi.json](#)

CloudMLOps is a three-tier AI document summarization platform.

- **Presentation tier** - React single page application
- **Business tier** - this FastAPI service (authentication, extraction, summarization)
- **Data tier** - PostgreSQL

Upload a PDF, DOCX or TXT file (or paste raw text) and the FLAN-T5 model produces an abstractive summary. Every summary is stored so it can be searched, re-read, downloaded and rated.

Authentication

Call **POST** `/api/auth/login`, then send the returned token as **Authorization: Bearer <token>** on every other request.

[CloudMLOps - Website](#)

MIT

Authorize



System Health and readiness probes.



GET

`/health` Service health check



Authentication Registration, login, logout and profile.



POST

`/api/auth/register` Register a new account



POST

`/api/auth/login` Exchange credentials for a JWT access token



POST

`/api/auth/logout` Revoke the current access token



Swagger UI served from the deployed FastAPI backend.

Results & Outcomes

What is working today

Live

AWS App Runner

OK

DB connected

171

Tests pass

UC-12

Model promote

- 171 pytest cases — auth, uploads, AI paths, admin, migrations.
- Production health: database connected; summarizer ready on AWS.
- Terraform + GitHub Actions: repeatable deploy pipeline.
- Model lifecycle: ROUGE evaluation, approve, promote (UC-12 / UC-16).

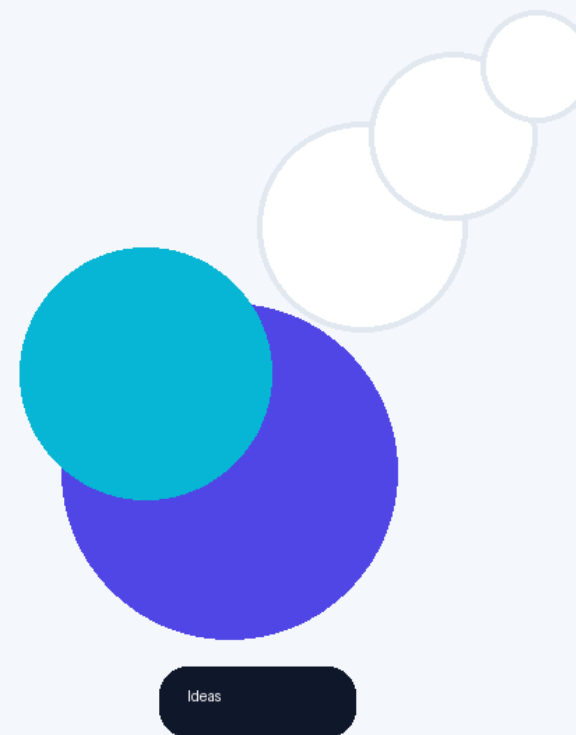


Ship to Production

Future Scope

Planned improvements (not yet implemented)

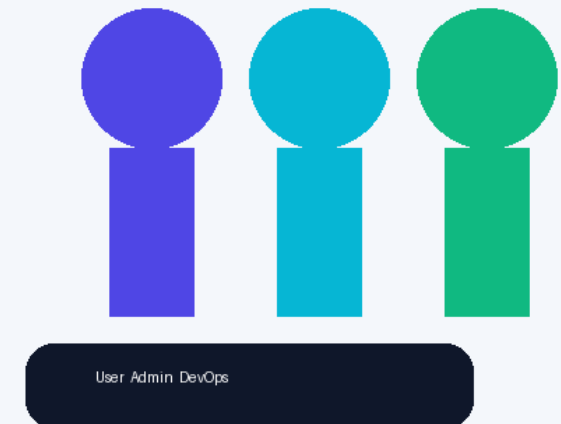
- GPU App Runner or SageMaker for full FLAN-T5 at scale.
- CloudWatch alarms + email/SNS when services degrade.
- Richer offline benchmarks when extra compute is available.



Conclusion

In one minute

- Built a real cloud app — not just slides — with live URLs you can click.
- Solved document overload with extract, summarize, store, and rate.
- Proved engineering quality with tests, CI, and AWS MLOps patterns.



Thank You

CLOUDMLOPS

CIS 5690 - Cloud MLOps

Questions & Discussion

[Open Live Demo](#)